1. What is Python?

Python is a **high-level, interpreted programming language** known for its simple and readable syntax. It supports multiple programming paradigms such as procedural, object-oriented, and functional programming. Python is widely used in web development, automation, data science, and testing.

---

2. What are the key features of Python?

Python is easy to learn because of its simple syntax and readability. It is platform independent, open-source, and has a large standard library. Python also supports object-oriented programming and automatic memory management.

---

3. What is an interpreted language?

An interpreted language executes code **line by line using an interpreter** instead of compiling the entire program beforehand. This makes debugging easier because errors can be identified quickly during execution.

---

4. What is a dynamically typed language?

In Python, you do not need to declare the data type of a variable explicitly. The type is determined automatically at runtime based on the value assigned to the variable.

---

5. What is PEP 8?

PEP 8 is the **official style guide for writing Python code**. It provides guidelines for code formatting, naming conventions, indentation, and structure to improve readability and maintainability.

---

6. What are Python keywords?

Python keywords are **reserved words that have predefined meanings in the language** and cannot be used as variable names. Examples include `if`, `else`, `while`, `for`, `class`, and `def`.

---

7. What are variables in Python?

Variables are used to **store data values in memory**. In Python, variables are created automatically when a value is assigned without specifying the data type.

---

8. What are identifiers?

Identifiers are the **names given to variables, functions, classes, and modules** in Python. They must start with a letter or underscore and cannot contain spaces or Python keywords.

---

9. What are comments in Python?

Comments are used to explain the code and improve readability. Python ignores comments during execution, and they are typically written using the `#` symbol.

---

10. What is indentation in Python?

Indentation refers to the spaces at the beginning of a line of code. Python uses indentation to define code blocks instead of curly braces, making code structure clearer.

---

11. What are built-in data types in Python?

Python provides several built-in data types such as **int, float, str, list, tuple, set, dictionary, and boolean**. These data types help store and manipulate different types of data.

---

12. What is the difference between list and tuple?

A list is a mutable data structure, meaning its elements can be modified after creation. A tuple is immutable, which means its elements cannot be changed once it is created.

---

13. What is a dictionary in Python?

A dictionary is a **collection of key-value pairs** where each key is unique. It allows fast data retrieval and is commonly used to store structured data.

---

14. What is a set in Python?

A set is an unordered collection of unique elements. It automatically removes duplicate values and is useful for operations like union, intersection, and difference.

---

15. What is the difference between mutable and immutable objects?

Mutable objects can be modified after creation, such as lists and dictionaries. Immutable objects cannot be changed once created, such as strings, tuples, and integers.

---

16. What are numeric data types in Python?

Python supports three main numeric types: **int, float, and complex**. Integers represent whole numbers, floats represent decimal numbers, and complex numbers contain real and imaginary parts.

---

17. What is type casting?

Type casting is the process of converting one data type into another. Python provides built-in functions like `int()`, `float()`, and `str()` for this conversion.

---

18. What is the difference between str() and repr()?

`str()` returns a readable string representation of an object, mainly for users. `repr()` returns a detailed representation of the object, mainly for debugging and developers.

---

19. What is None in Python?

`None` represents the absence of a value or a null value. It is often used to initialize variables or indicate that a function does not return any value.

---

20. What is the difference between is and ==?

`==` checks whether the values of two variables are equal. `is` checks whether both variables refer to the same object in memory.

---

21. What are arithmetic operators?

Arithmetic operators perform mathematical calculations such as addition, subtraction, multiplication, division, modulus, and exponentiation.

---

22. What are comparison operators?

Comparison operators are used to compare two values and return a boolean result. Examples include `==`, `!=`, `>`, `<`, `>=`, and `<=`.

---

23. What are logical operators?

Logical operators combine multiple conditions. Python provides three logical operators: `and`, `or`, and `not`.

---

24. What are assignment operators?

Assignment operators are used to assign values to variables. Examples include `=`, `+=`, `-=`, `*=`, and `/=`.

---

25. What are membership operators?

Membership operators check whether a value exists in a sequence. Python provides `in` and `not in` for this purpose.

---

26. What is an if statement?

An `if` statement is used for decision-making in Python. It executes a block of code only when a specified condition evaluates to true.

---

27. What is the difference between if, elif, and else?

`if` checks the first condition, `elif` checks additional conditions if the previous ones fail, and `else` executes when none of the conditions are true.

---

28. What is a for loop?

A `for` loop is used to iterate over a sequence such as a list, tuple, string, or range. It allows repeated execution of a block of code.

---

29. What is a while loop?

A `while` loop repeatedly executes a block of code as long as the condition remains true. It is commonly used when the number of iterations is not known beforehand.

---

30. What is the difference between break and continue?

`break` immediately stops the loop execution and exits the loop. `continue` skips the current iteration and moves to the next iteration of the loop.

---

31. What is a function?

A function is a reusable block of code that performs a specific task. Functions help organize code and reduce repetition.

---

32. How do you define a function in Python?

Functions are defined using the `def` keyword followed by the function name and parentheses. The function body contains the code that executes when the function is called.

---

33. What are arguments and parameters?

Parameters are variables defined in the function definition. Arguments are the actual values passed to the function when it is called.

---

34. What are positional and keyword arguments?

Positional arguments are passed in the same order as parameters in the function definition. Keyword arguments specify the parameter name while passing the value.

---

35. What are default parameters?

Default parameters allow a function to use a predefined value if no argument is provided during the function call. This makes functions more flexible.

---

36. What is a lambda function?

A lambda function is a small anonymous function defined using the `lambda` keyword. It is commonly used for short operations that do not require a full function definition.

---

37. What is recursion?

Recursion occurs when a function calls itself to solve a problem. It is often used in problems that can be divided into smaller similar subproblems.

---

38. What is list comprehension?

List comprehension provides a concise way to create lists in a single line of code. It combines loops and conditional statements into a compact syntax.

---

39. What are dictionary methods?

Dictionary methods are built-in functions used to manipulate dictionary data. Common methods include `keys()`, `values()`, `items()`, and `get()`.

---

40. What are string methods?

String methods are functions used to manipulate strings. Examples include `upper()`, `lower()`, `split()`, `replace()`, and `strip()`.

---

41. What is Object-Oriented Programming?

Object-Oriented Programming (OOP) is a programming paradigm based on objects and classes. It helps organize code into reusable and structured components.

---

42. What is a class in Python?

A class is a blueprint for creating objects. It defines properties (attributes) and behaviors (methods) that objects created from the class will have.

---

43. What is an object?

An object is an instance of a class. It represents a real-world entity and contains both data and methods that operate on that data.

---

44. What is inheritance?

Inheritance allows one class to acquire the properties and methods of another class. It helps in code reuse and creating hierarchical relationships.

---

45. What is encapsulation?

Encapsulation is the concept of restricting direct access to certain data and methods. It protects data by allowing access only through defined methods.

---

46. What is exception handling?

Exception handling is used to manage runtime errors using `try`, `except`, and `finally` blocks. It prevents the program from crashing when errors occur.

---

47. What is the difference between syntax error and runtime error?

A syntax error occurs when the code violates Python's grammar rules. A runtime error occurs during program execution even though the syntax is correct.

---

48. What is a module?

A module is a file containing Python code such as functions and variables. Modules help organize code and promote reuse across multiple programs.

---

49. What is a package?

A package is a collection of multiple modules organized in directories. It helps structure large Python projects and manage related modules efficiently.

---

50. What is `__name__ == "__main__"`?

This condition checks whether the Python file is executed directly or imported as a module. Code inside this block runs only when the file is executed directly.
